AutoBench

Architecture & design overview

v1.36 · OpenShift and KinD · 2026-10-12

Outline

6 sections; the numbers are slide numbers

1 The design

what it is, its decisions, auth, its boundary

3 What it is, seven decisions
4 The design motif
5 The two-token auth model
6 What the Service can enact

2 Architecture & deployment

the system, the workload, interception, installing

7 The architecture
8 Inside the workload
9 How interception is wired
10 The double bypass
11 Installing & uninstalling

3 Running a benchmark

the REST lifecycle, the images, one task in time

12 Catalog & run lifecycle
13 What each image bakes in
14 One task, end to end
15 Who decides inside a task

4 The three benchmarks

what each measures, the traps, what a leg costs

16 The difficulty ladder
17 What each stresses
18 The three compared
19 Five traps
20 Picking a benchmark
21 What each leg costs

5 Cost

the model, the rate card, six efficiency charts

22 The cost model
23 The rate card
24 Which way the dollars are wrong
25 Chart: money vs tokens
26 Chart: share of the bill
27 Chart: model choice
28 Chart: where the bill goes
29 Table: cost per success
30 Chart: the judge's bill

6 Results (v1.36)

the 12-run matrix, OpenShift vs KinD, plugins (v1.35)

31 The 12-run matrix
32 What each band established
33 What the 12 runs measured
34 OpenShift vs KinD
35 What AuthBridge costs
36 Why the plugin result holds

What AutoBench is, and the seven decisions that shape it

A pure-Python, HTTP-only service that deploys, runs and grades agent benchmarks on Rossoctl clusters

Deploys a benchmark's MCP tool and A2A agent, runs the evaluation, reads the results back and exports them, all over HTTP.

Objectives

- Automate the benchmarking lifecycle
- Share run results across users and clusters
- One Service, many clusters: the caller's JWT `iss` picks the instance
- Asynchronous, parallel benchmark runs
- Secure, auditable, resilient

Three benchmarks

- **gsm8k**, single-turn: one prompt in, one answer out
- **tau2**, multi-turn: a user-simulator LLM plays the customer
- **appworld**, long-horizon: many API calls across simulated apps

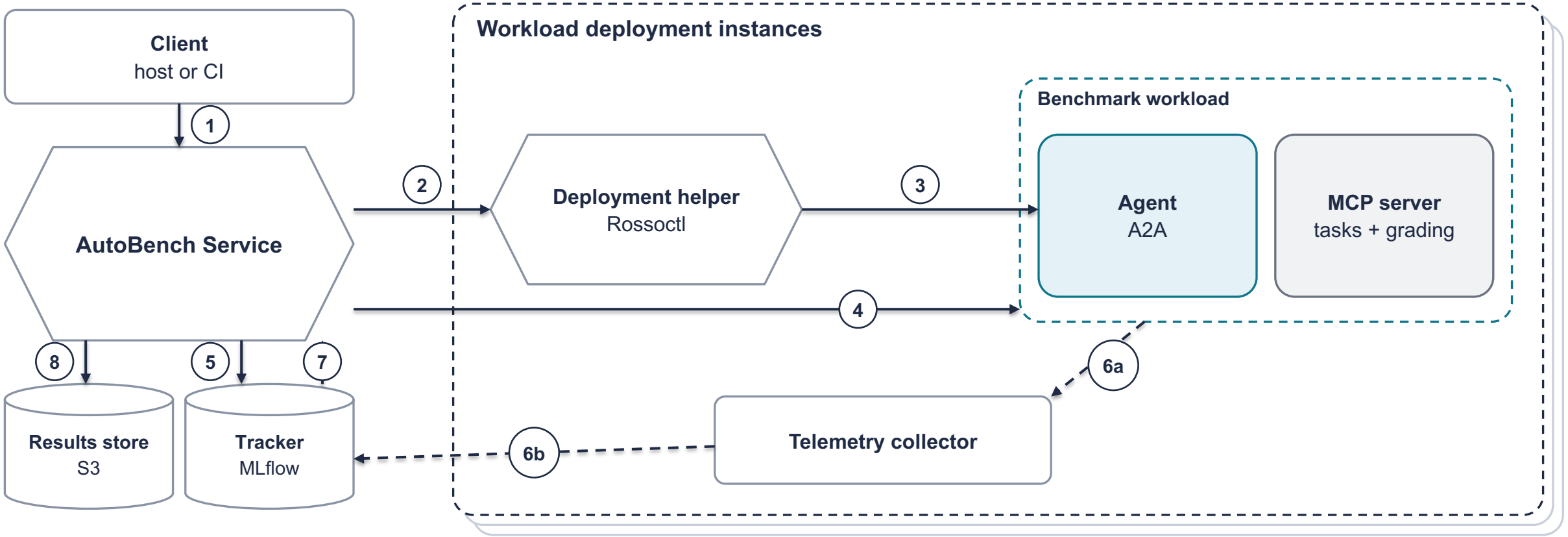
Client contract

- A hostname and `Authorization: Bearer <caller JWT>`, the same on every cluster

Decision	Why
Pure-Python, HTTPS-only to Rossoctl	No shell-out: a smaller image, no injection surface, structured errors, testable
Two-token auth	The caller's JWT attributes and routes, and is never forwarded; the Service mints its own token
`iss` is the trust anchor	The issuer selects the instance: no instance argument, so no confused-deputy escape
Per-request ROPC login	A fresh Service token on every request: no expiry handling
One config file per issuer	Rossoctl URL, `benchmarker` credential, MLflow, S3, an optional Keycloak backchannel
Enact only what the API allows	Deploy, run, report, export, plugin presets; workload Secrets are prechecked (424), never ignored
MLflow with the Service, S3 in the cloud	Traces live beside the Service; S3 is the shared, cross-cluster sink for results

One client, one Service, N workload deployments

The shape of the system before the wiring: who drives whom, and where every result lands



1 Client → Service
2 Service → deployment helper: create the workload
3 helper → workload
4 Service → workload: run and monitor
5 Service → tracker: emit the trace
6a agent → collector · 6b collector → tracker
7 tracker → Service: read the records back
8 Service → results store: export the artifacts

The caller's token is never forwarded upstream

Two tokens: one says who is asking, the other is what the Service acts with

Caller JWT (inbound)

Presented by the client as `Authorization: Bearer`.

- Used only to attribute (`preferred_username`) and route (`iss` → instance)
- Signature validated against the issuer's JWKS
- `aud` and `exp` deliberately lenient in the dev/ops context
- **Never forwarded to Rossoctl**

`benchmarker` is special

- A JWT whose `preferred_username` is `benchmarker` authorizes `GET/PUT /config`

Service token (outbound)

Minted per request by ROPC, with the instance's `benchmarker` credential.

- Always fresh, so no expiry handling
- Presented to Rossoctl for every cluster-facing operation
- **The Service acts under its own identity, not the caller's**

Backchannel split (`iss` ≠ the host dialled)

- When the `iss` host is unreachable from the pod (in-cluster KinD), JWKS and tokens come from a backchannel URL; `iss` is matched, never dialled

Implication

At the Rossoctl and cluster layer every action is the Service's identity, so per-user attribution and audit live in the Service, keyed on (`iss`, `preferred_username`).

What the Service can and cannot enact

The HTTP-only boundary, made explicit: anything else is reported, never silently ignored

Enacts over HTTP

- Deploy the MCP tool and A2A agent (CPU and memory, image, env)
- Swap the model at deploy time (a per-experiment agent)
- `authbridge_enabled` injects the sidecar (layer 2)
- `plugin_preset`, `plugins`, `on_error` set AuthBridge layer 3
  - the operator renders the pipeline ConfigMap
- Run benchmark sessions; collect pass/fail and latency
- Emit and read MLflow traces; export to S3
- Its own config, MLflow and S3, via `PUT /config` (`benchmarker` only)

Out-of-band: reports it, does not do it

- Cluster Secrets (`hf-secret`, `openai-secret`)
  - the operator provisions them; the run precheck returns 424 naming the missing one
- AuthBridge cluster config: `ibac.judgeEndpoint`, `judgeModel`
  - in the platform ConfigMap, not the agent API
- Any cluster-level API call: Rossoctl does those server-side

The principle

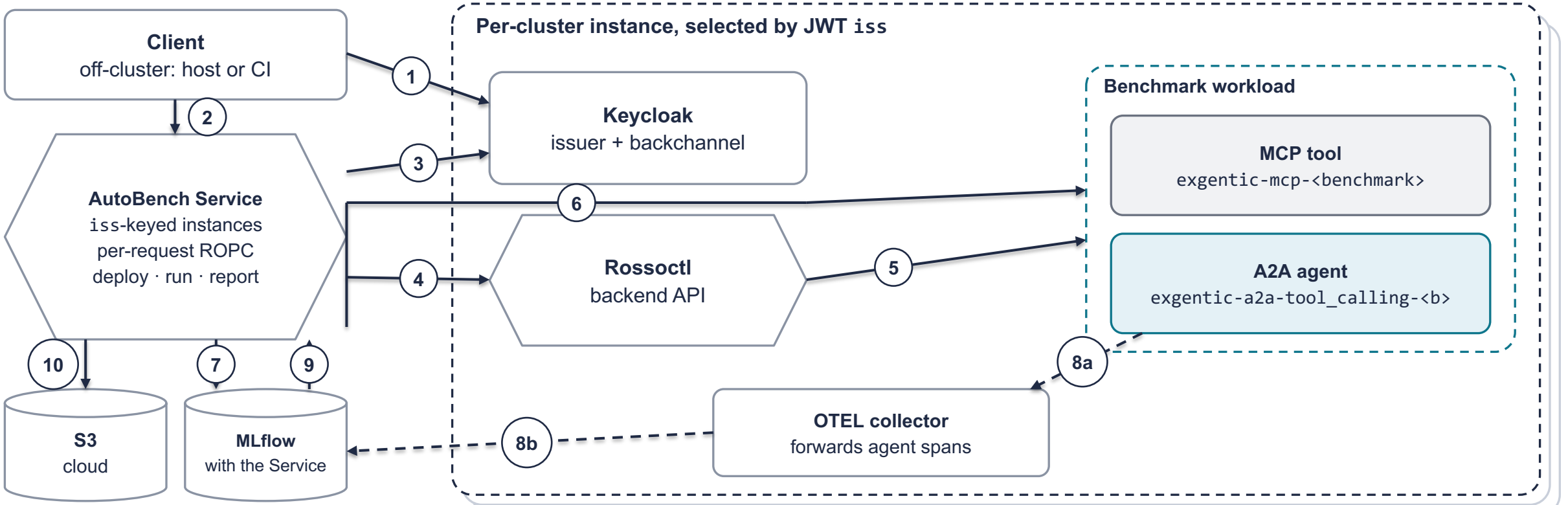
- An un-enactable request is prechecked (424) or rejected (422) with an actionable reason, never dropped

Cluster-agnostic by construction

The same chart on KinD and OpenShift; the `iss`-keyed instance decides where the workloads run.

The architecture: one Service, a per-cluster instance selected by `iss`

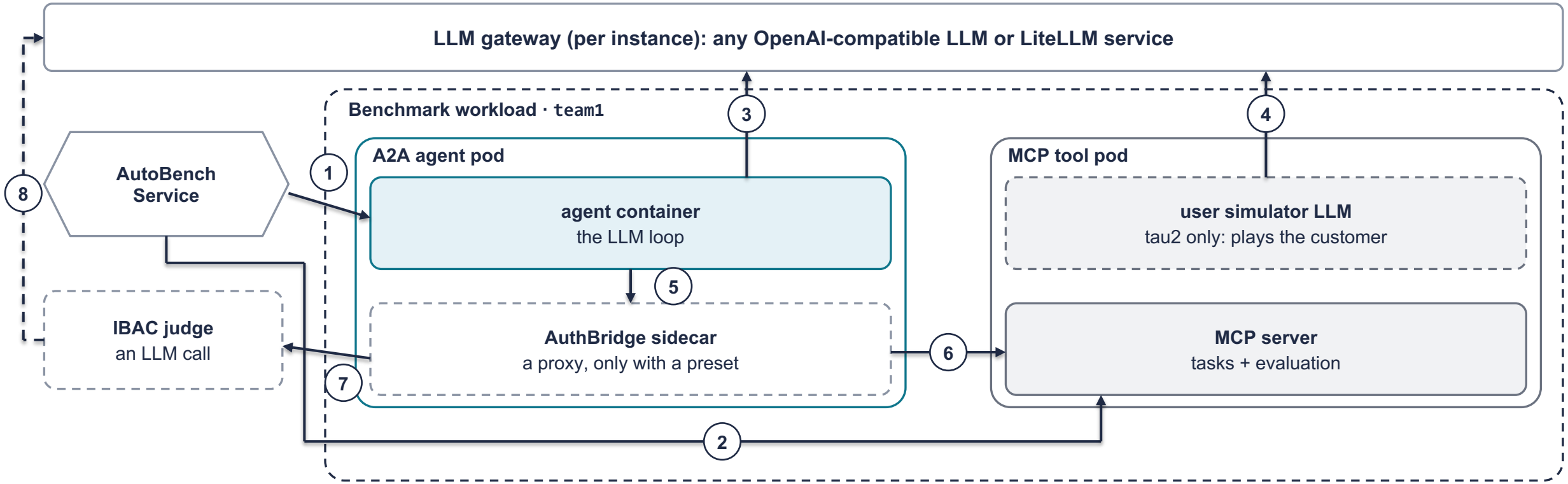
Client, Service and the cluster-specific Keycloak, Rossoctl and workload; MLflow sits with the Service, S3 in the cloud



1 Client logs in to Keycloak (ROPC): the caller JWT
2 Client → Service: `Bearer <caller JWT>`
3 Service validates the JWT (JWKS) and mints its own token
4 Service → Rossoctl: deploy, list agents and tools
5 Rossoctl creates the MCP tool and A2A agent
6 Service drives the MCP session and the A2A call
7 Service emits the `Agent.Session` trace → MLflow, first
8a agent → OTEL collector · 8b collector → MLflow (optional)
9 Service reads the traces back from MLflow
10 Service exports `run.json`, `report.*` to S3

Inside the workload: the fixed parts and the optional ones

Dashed = present only in some configurations; with no preset, flows 5 and 6 are one direct agent → MCP call



benchmark	LLMs per task	tool calls
gsm8k	1 (agent)	~1
tau2	2 (+ user simulator)	~11
appworld	1 (agent)	~15
+ any preset	+1 judge call per action	unchanged

1 Service → agent: `send_prompt`, once per task
2 Service → MCP server: the session, and the verdict
3 agent → gateway: N chat calls per task
4 tau2: the user simulator is a second LLM
5–6 with a preset, MCP calls go via the sidecar
7 sidecar → judge per action · 8 the judge is an LLM call

The sidecar sees traffic only because of two env vars

Interception is a blanket forward proxy plus an allowlist, not a tool-aware hook

`HTTP_PROXY = HTTPS_PROXY = 127.0.0.1:8081`

The operator injects it into the agent pod when AuthBridge is on. That address is the sidecar, so by default every outbound HTTP call is captured.

`no_proxy`: gateway, collector, Keycloak

Instance config: the hosts that stay direct. The only reason the agent's own chat calls (flow 3) skip the sidecar.

`judge_inference: false`

IBAC's default: even proxied inference is not judged. Both this and `no_proxy` would have to change for model calls to be authorized.

Through the sidecar

- agent → MCP tool calls (flows 5 → 6): the interception point
- `tools/call` is `isAction=true`: one judge call each (flow 7)
- inbound A2A `message/stream` is `isAction=false`: the Service's `send_prompt` is logged, never judged

Direct: the plugins never see it

- agent → LLM gateway (flow 3): named in `no_proxy`
- Service → MCP server (flow 2): a different pod, not dialled through the proxy
- MCP tool → gateway (flow 4, tau2's user simulator): a different pod

The only test that settles it

Count judge completions across a run. IBAC logs nothing of its own, so a healthy sidecar log proves nothing; and a zero means something only if real tool calls ran.

A ready sidecar proves nothing: the double bypass we shipped

Four steps, each defensible alone; together the judge was called zero times, on both clusters

1 The Service
`workload_llm.disable_proxy: true` makes it inject `HTTP_PROXY=""` first

→

2 The operator
adds a var only when it is absent: skips `HTTP_PROXY`, still sets `HTTPS_PROXY`

→

3 The agent
`MCP_URL` is `http://`, so tool calls take the empty `HTTP_PROXY` and go direct

→

4 The allowlist
`no_proxy` carried `.svc.cluster.local`: the same calls excluded a second time

Result
Sidecar ready, pipeline ConfigMap rendered, plugins loaded, and the judge called ZERO times: a plausible config that takes interception back out of the path.

The working config
- leave `disable_proxy` off
- keep the named hosts in `no_proxy`
- drop the wildcards (`.svc`, `.svc.cluster.local`)

How to verify it
- count judge completions across a run, never sidecar logs
- `initialize` and `tools/list` are `isAction=false`: never judged
- so a zero count needs real tool calls behind it

Installing and uninstalling: the scripts cover one cluster

A cross-cluster service–workload setup is supported by the Service, and left to you

`reference/autobench-install.sh`

- One env file, one command, on KinD or OpenShift
- Preflight first: 0 failures, or it stops
- MLflow read path, instance file, Secret, Helm, restart
- Verifies `/healthz`, the S3 block, then preflight with the MLflow round trip
- `--ibac-judge` adds the judge for the plugin legs

`reference/autobench-uninstall.sh`

- Workloads first, through the Service; then `helm uninstall`
- Verifies every owned object is gone and the judge fields restored
- Removes only what the installer recorded creating

Scope: one cluster

- The Service and its workloads on the same cluster
  - an instance file for that cluster's own `iss`; endpoint templates `null`

Cross-cluster service–workload: supported, not scripted

- an instance file keyed to the workload cluster's `iss`: its Rossoctl URL, endpoint templates, `workload_otel`, the Service-side MLflow
- a collector on the workload cluster, forwarding to the Service's MLflow
- the IBAC judge on the workload cluster
- callers take their token from the workload cluster's Keycloak

The Service needs no change for it: only provisioning, which the scripts do not yet do.

The catalog, and the REST calls that drive a run

Three benchmarks in a static registry; six calls from deploy to the artifacts

Catalog (static registry)

gsm8k
single-turn; needs `hf-secret` and `openai-secret`

tau2
multi-turn; the user-simulator LLM runs server-side in the MCP pod; raise the timeout

appworld
long-horizon across simulated apps; raise `task_timeout_seconds`

Lifecycle (REST)

call	what it does
`POST /benchmarks/{n}/deploy`	create the MCP tool and A2A agent → 201
`GET /benchmarks/{n}/status`	poll until `tool_ready` and `agent_ready`
`POST /benchmarks/{n}/runs`	precheck → 202 `run_id` (409 not deployed, 424 a Secret missing)
`GET …/runs/{id}`	poll pending → running → succeeded or failed
`GET …/runs/{id}/report`	MLflow records and artifacts (409 if MLflow is unset)
download from S3	`run.json`, `report.*`, `token_report.*`, `span_report.*`, `manifest.json`

The image carries the benchmark; `tool_env` adds credentials

What each image bakes in, and the two env vars per benchmark that it does not

	MCP tool image	baked into the image	added by `tool_env`
gsm8k	`exgentic-mcp-gsm8k`	the HuggingFace loader, pinned to main/test: those 1,319 rows, fetched at pod start	`HF_TOKEN` (from `hf-secret`), plus `EXGENTIC_SET_BENCHMARK_RUNNER=direct`
tau2	`exgentic-mcp-tau2`	tau2-bench at v0.1.3, its retail domain (114 tasks), and a user-simulator LLM	`OPENAI_API_KEY` (from `openai-secret`) and an action timeout of 1000: the simulator makes its own calls
appworld	`exgentic-mcp-appworld`	the whole app-suite sandbox at upstream `edc96012`, behind a custom tool-per-API adapter; test_normal split	nothing beyond `BENCHMARK_NAME`

One agent image serves all three

`exgentic-a2a-tool_calling:latest` is the only agent; per benchmark it gains a name suffix. The benchmark lives in the MCP pod; the subject is the same binary.

The LLM base is never baked in

`OPENAI_API_BASE` comes per deploy from the instance's `workload_llm.api_base`; a deploy with no gateway configured is rejected (422), not defaulted.

Two traps in the last column

- appworld rejects the action-timeout override tau2 needs, and crashes at start: the env is per benchmark, not a shared default.
- `hf-secret` must exist for gsm8k although the dataset is public, or the MCP pod sits in `CreateContainerConfigError`. A missing Secret surfaces as 424 on the precheck.

One task, end to end: everything the agent does is one A2A turn

The same system in time, with the span each step appears as

who	step	span
Service	`create_session(task_id)` → MCP pod	`MCP.CreateSession`
Service	`send_prompt` → agent: the task text, `session_id` in A2A metadata; once per task	`Agent.Call`
agent	`connect_mcp`: `tools/list` → MCP pod (per task; ~40–50 ms)	`connect_mcp`
agent	`create_agent` (~12–15 ms warm, up to 5.2 s on a cold pod)	`create_agent`
agent	`initial_observation`: local, no I/O, ~70 µs	never counted
agent	the loop: chat → `execute_tool` → MCP pod → observation → chat → …	`chat` · `execute_tool`
agent	returns its final message when the model asks for no further tool	the runner discards it
Service	`evaluate_session` → MCP pod: the verdict, off the session's final state	`Evaluator.Evaluate`
Service	`delete_session` → MCP pod, in a `finally`	no span

One A2A turn per task

`Agent.Call` minus the agent's `POST /` span is the Service's own cost: a median 16.0 ms of a 5.7 s gsm8k task, 19.4 ms of a 104 s tau2 one.

The Service issues no tool call

Its MCP traffic is the session lifecycle. `execute_tool` is the agent's call; the Service never learns a tool's name.

The session id joins the halves

It rides in A2A metadata, never the prompt: the agent's tool calls mutate the session the Service grades, so prose alone fails a gsm8k task.

Nobody scripts a task's trajectory: the model produces it, step by step

What each component defines, and what it does not

component	defines	does NOT define
MCP pod	the task list; each task's initial state and instruction; the tool surface; the evaluator	any ordering: it answers calls, never asks
agent runtime	the loop (model → tool → observation → model) and when to stop	which tool, with which arguments
the Service	one prompt per task (no tool instructions), the session lifecycle, the timeout, the telemetry	anything about the trajectory
the model	every step	—

Selection is deterministic; execution is not

Same task, model and platform, two legs: tau2 task 9 took 12 chat / 12 tool in one and 11 / 11 in the other. 12 of 20 repeated tau2 tasks and 8 of 8 appworld differ; all 20 gsm8k agree.

The stop signal comes from the MCP pod

No terminal tool name exists in the Service. gsm8k calls `submit` 172/172; appworld `finish` 41/47; tau2 has no terminal tool, and 60/60 end on a message.

Why it matters downstream

Grading reads the final state, never a reference trajectory: per-task cost and latency are distributions, hence the CV columns in every report.

Three rungs of a ladder, each ~an order of magnitude dearer

Per task, measured on our own v1.36 runs, both platforms pooled; not the benchmarks' published papers

	gsm8k	tau2	appworld
model (the default)	Azure/gpt-5-mini	aws/claude-sonnet-5	gemini-2.5-pro
its rate, in / out $ per 1M	0.25 / 2.00	1.52 / 7.60	1.25 / 10.00
what it tests	multi-step arithmetic	multi-turn dialogue + tools	long-horizon app automation
task rows measured	172 of 172	60 of 60	47 of 50 (3 timed out)
pass rate	0.98	0.83	0.00
input / output tokens	342 / 177	89,122 / 2,078	268,880 / 26,382
cost, at those rates	$0.00054	$0.151	$0.600
LLM / tool calls	1.1 / 1.1	11.2 / 11.2	28.0 / 13.9
median / slowest task*	5.7 s / 19 s	121 s / 214 s	298 s / 581 s
task pool	1,319 (main/test)	114 (retail)	168 (test_normal)

Read the model row before any dollar figure

The rungs do not run the same model, so no cost here transfers to another one. A tau2 task is ~260× a gsm8k task's input tokens and 279× its dollars; appworld ~790× and 1,108×. *Latency on a slow gateway day: 2–3× v1.35's per call.

What each benchmark stresses, and what a result from it tells you

Why all three are in the matrix

gsm8k: the canary

- One prompt in, one answer out: ~1 LLM call and 1 tool call
- A failure means infrastructure: deploy, auth, LLM reach, telemetry
- Saturates near 1.0, so it cannot rank models
- Task 0 costs 320 input tokens on every cluster: proof two environments compare
- A 50-task leg is ~24 K tokens, less than one appworld task

tau2: the discriminator

- A server-side user-simulator LLM plays the customer; ~11 tool calls
- Retail domain (114 tasks), the library default, recorded nowhere
- Model choice dominates: 0.1 on gpt-5-mini, 0.9 on claude-sonnet-5, same 10 tasks
- Nondeterministic: the same 10 tasks scored 0.70 and 0.90 on two clusters
- 23% of wall time is the simulator, whose tokens no report of ours sees

appworld: the stress test

- Chores across simulated apps: ~28 LLM calls, ~14 tool calls, ~270 K input
- Graded by programmatic tests over the final app state, no LLM judge
- 0.00 is honest: runs complete and tokens record; the job is not done
- 41 of 47 tasks call `finish`; the assertions disagree
- Tool shortlisting: two model calls per tool call, 67% of the input

They differ in the shape of a task, not just its size

What a task is, how it ends, and what a result from it is worth

	gsm8k	tau2	appworld
a task is	one grade-school word problem, answered in text	one retail customer-service conversation with a simulated user	one multi-app scenario automated through an API surface
turns	1 model call, ~1 tool call	~11 calls alternating with the simulator	~28 calls, each re-sending the whole conversation
ends when	the answer is emitted	the dialogue resolves, or policy is violated	the goal state is reached, or the 600 s timeout kills it
scored by	exact numeric match	completion and policy compliance	appworld's state assertions, all or nothing
it tests	that the plumbing works	that the agent holds state and uses tools under a policy	that the agent survives long horizons
a result is worth	a go/no-go on infrastructure	a real model comparison, with headroom both ways	a stress signal: what breaks, not who is better
watch out for	1.0 proves nothing about the agent	the simulator is billed but not in our telemetry	a killed task leaves no report row

Five things that mislead in reading the numbers

Every one of these cost us a wrong conclusion first

1 `pass_rate` = evaluated passes / total

A task that errors before evaluation counts as not passed: check the error column.

2 The `llm` column changed meaning between agent versions

Agents up to exgentic 0.3.5.dev131 sent a counted probe; the current one sends none (absent across all 2,179 chat spans). Never compare across it.

3 Tiny tokens are lost telemetry, and `tokens == 0` will not catch it

Use the structural test: `llm` ≤ 1 with `tool` ≥ 2 is impossible.

4 Output varies more than input, in most runs

OUT CV > IN CV in 17 of 22 multi-task legs; only appworld is input-led. Read the CV.

5 Task selection is deterministic

The first `max_tasks` tasks: the same `task_id` is the same task on every cluster.

Pick a benchmark by what it is for

Dollars at the rate card on the cost slides; v1.36 measured costs

if you want to …	use	costs
check a cluster, deploy, auth or telemetry path works	gsm8k, 1–10 tasks	< $0.01
exercise concurrency and volume cheaply	gsm8k, 50 tasks at p=4	$0.02
compare models meaningfully	tau2: it discriminates, gsm8k saturates	$1.62 / 10
stress long contexts, long tasks, timeouts	appworld	$2.10–2.25 / 5
get a fast signal that nothing regressed	gsm8k: if it fails, fix infrastructure	< $0.01

Budget by benchmark, not by task count

The eight gsm8k legs together are 0.2–0.3% of the matrix's bill; appworld's two legs are 75% of it on both platforms. A 50-task gsm8k leg costs 2 cents, about a thirtieth of a single appworld task.

What each leg costs: the same work, close to the same bill

Tokens and dollars per leg, v1.36, OpenShift (OCP) and KinD

leg	tok OCP	$ OCP	tok KinD	$ KinD
#1 gsm8k, 1 task	406	$0.0003	470	$0.0004
#2 gsm8k, 10 tasks	5.2 K	$0.005	4.8 K	$0.004
#3 gsm8k, 50 tasks p=4	24 K	$0.020	25 K	$0.022
#9 tau2, 10 tasks	0.97 M	$1.62	0.99 M	$1.64
#10 tau2, 20 tasks p=4	1.76 M	$2.92	1.75 M	$2.89
#11 appworld, 5 tasks	1.10 M	$2.24	1.01 M	$2.11
#12 appworld, 20 tasks p=4	5.86 M	$11.83	5.91 M	$12.02
all 12 legs	**9.7 M**	**$18.67**	**9.7 M**	**$18.70**

Within 0.2%, but not by law

The totals land within 0.2% ($18.67 / $18.70): the same request bodies through one gateway. appworld turn counts are nondeterministic, so size it on your cluster.

The totals understate it

- a task killed by its timeout burns tokens and leaves no report row
- tau2's user simulator is billed by the gateway and counted nowhere here
- a replayed completion re-reports usage never made upstream
- the IBAC judge's calls are not in the agent's spans

Money amplifies the difficulty ladder rather than tracking it

Per task, pooled over both platforms, at our gateway's posted rates

per task	gsm8k	tau2	appworld
model	gpt-5-mini	claude-sonnet-5	gemini-2.5-pro
LLM calls	1.11	11.18	28.02
total tokens	519	91,201	295,261
× gsm8k, in tokens	1×	176×	569×
cost	$0.00054	$0.151	$0.600
× gsm8k, in dollars	1×	279×	1,108×
cost of 100 tasks	$0.05	$15.13	$59.99
input share of tokens	66%	98%	91%
input share of cost	31%	90%	56%
time inside model calls	77%	64%	95%

The formula

cost per task = (input tokens × P_in + output tokens × P_out) / 1 M. Which model is cheaper is a property of the price list, not of the models.

Why money climbs faster

The harder benchmarks also run the dearer models: tau2 is 176× a gsm8k task in tokens but 279× in dollars, appworld 569× but 1,108×. A budget scaled off the token ratios is short by 1.6–1.9×.

Output is priced 4–8× input at every provider

The rate card: read off the gateway's admin UI on 2026-09-17, kept in `reference/model_prices.json`; posted rates, not an invoice

model (as `report.ndjson` records it)	provider	in $/1M	out $/1M	out/in	used by
Azure/gpt-5-mini-2025-08-07	azure	0.25	2.00	8.0×	gsm8k
gemini-2.5-pro	vertex_ai	1.25	10.00	8.0×	appworld
aws/claude-sonnet-5	bedrock	1.52	7.60	5.0×	tau2
Azure/gpt-4.1	azure	2.00	8.00	4.0×	leg #4, and the IBAC judge

That one fact explains the counter-intuitive results

A reasoning model's verbosity is charged at the expensive end, so a model can win on tokens and lose on the bill. Read the input share of cost, not of tokens.

Which way the dollars are wrong: one bias each way

The judge makes our totals too low; an unmodelled cache discount, too high

Too low: the IBAC judge is not in our telemetry

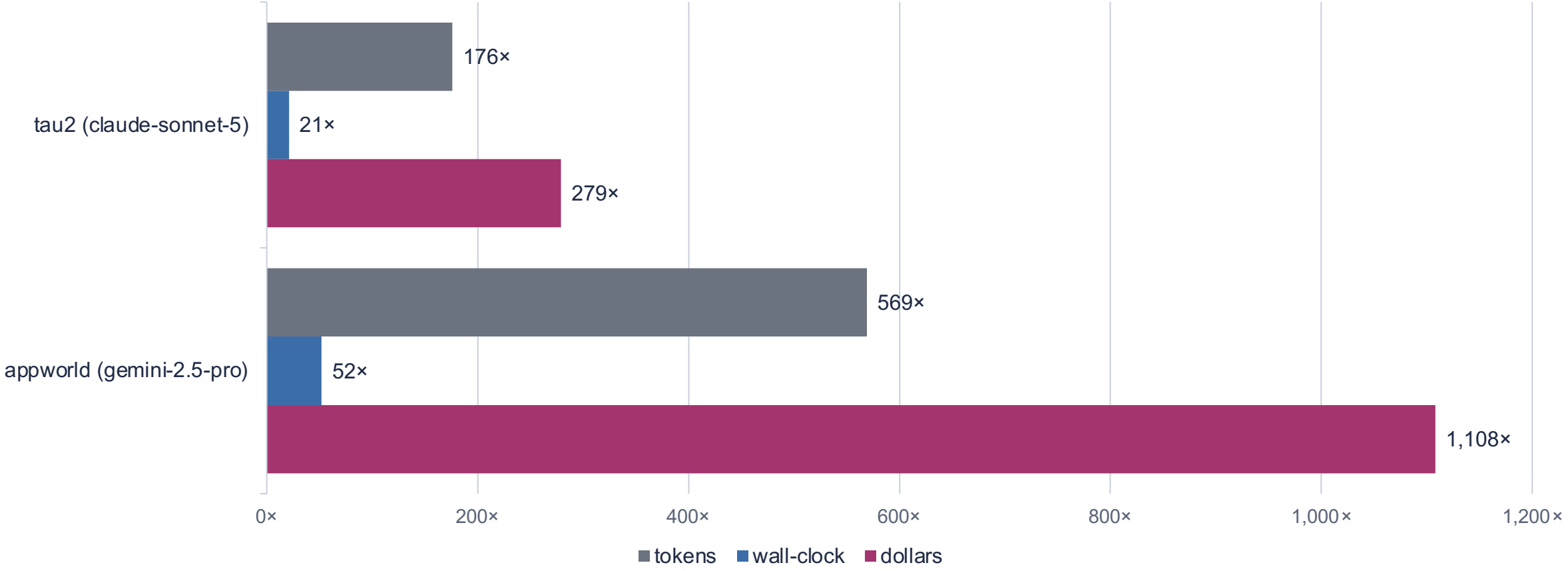
- ~1 completion per authorized tool call, on Azure/gpt-4.1, the dearest model on the card
- a fixed 1,577-character system prompt that does not shrink with the task
- ≥ $0.00111 per call: 2.5× the whole gsm8k task it authorizes
- 1.2–2.9× the agent's own bill on legs #6–#8
- tau2's user simulator is invisible the same way

Too high: an unmodelled cache discount

- the gateway reports cached input but publishes no cached-input rate
- `report.ndjson` stores one undifferentiated input count: no past run can be re-priced
- if cached input were free: tau2 floors at $0.0158 a task (not $0.151), appworld at $0.264 (not $0.600)
- gsm8k barely moves: caching pays off on a long re-sent prefix

Money climbs the ladder faster than tokens

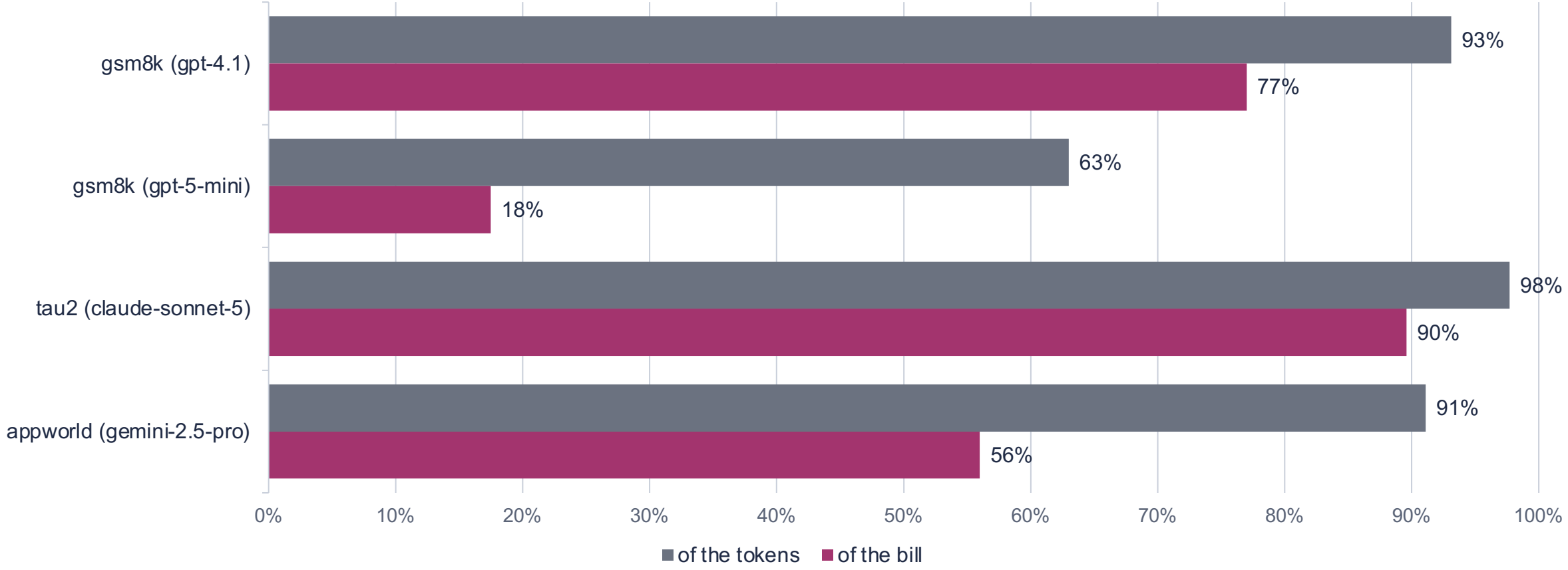
Multiple of one gsm8k task on gpt-5-mini per task; v1.36, both platforms pooled



Money climbs the difficulty ladder faster than tokens do: tau2 is 176× a gsm8k task in tokens but 279× in dollars, appworld is 569× a gsm8k task in tokens but 1,108× in dollars — so a budget scaled off the token ratios is short by 1.6–1.9×.

Read the share of the BILL, not the share of tokens

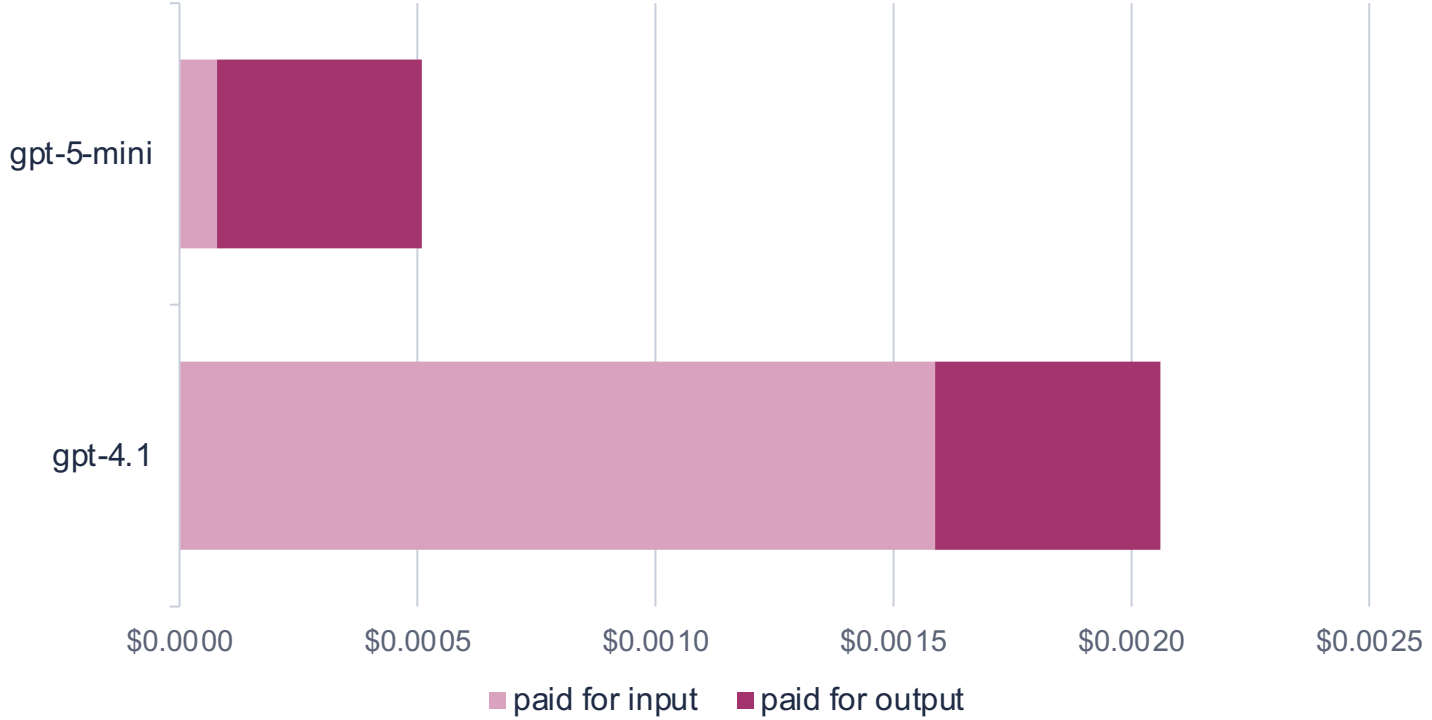
Input's share, % of a task's tokens against % of its bill; per benchmark and model



Output is priced 4–8× input on every model we use, so the token split misstates the bill every time: gsm8k on gpt-5-mini is 63% input by tokens and only 18% by cost.

Model choice is settled on the input side

Dollars per task, split into what input and output cost; the same 5 gsm8k tasks



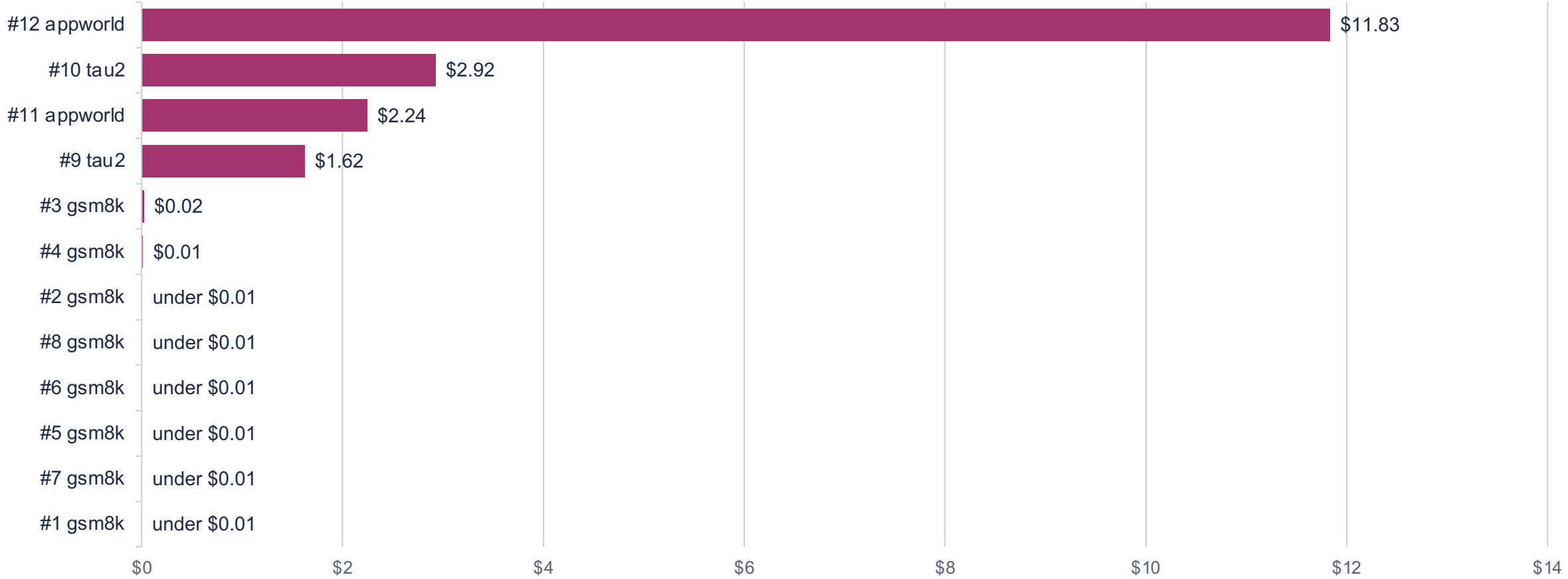
per task	gpt-5-mini	gpt-4.1
cost	$0.00051	$0.00206
input tokens	313	794
output tokens	216	59
pass rate	0.97	0.80

4.0× the cost per task, on identical work: the reasoning model answers in one call; gpt-4.1 takes ~3 tool round-trips.

gpt-4.1 costs 4.0× gpt-5-mini on the identical 5 gsm8k tasks, and the input side decides it alone: 20× the input bill, against an output bill the two models split within 9%.

Two legs of twelve are most of the bill

Dollars per leg, OpenShift v1.36: $18.67 for all twelve, the top two 79% of it



Two of the twelve legs are 79% of a matrix's bill, while all 8 gsm8k legs together are 0.3% of it — so budget by benchmark, not by task count.

Efficiency only means anything per SUCCESS

Dollars per task attempted and per task passed, pooled; a table, because the three span three orders of magnitude

benchmark	per task attempted	pass rate	per task PASSED
gsm8k	$0.00054	0.98	$0.00055
tau2	$0.15126	0.83	$0.18151
appworld	$0.59992	0.00	no finite cost: it passed nothing

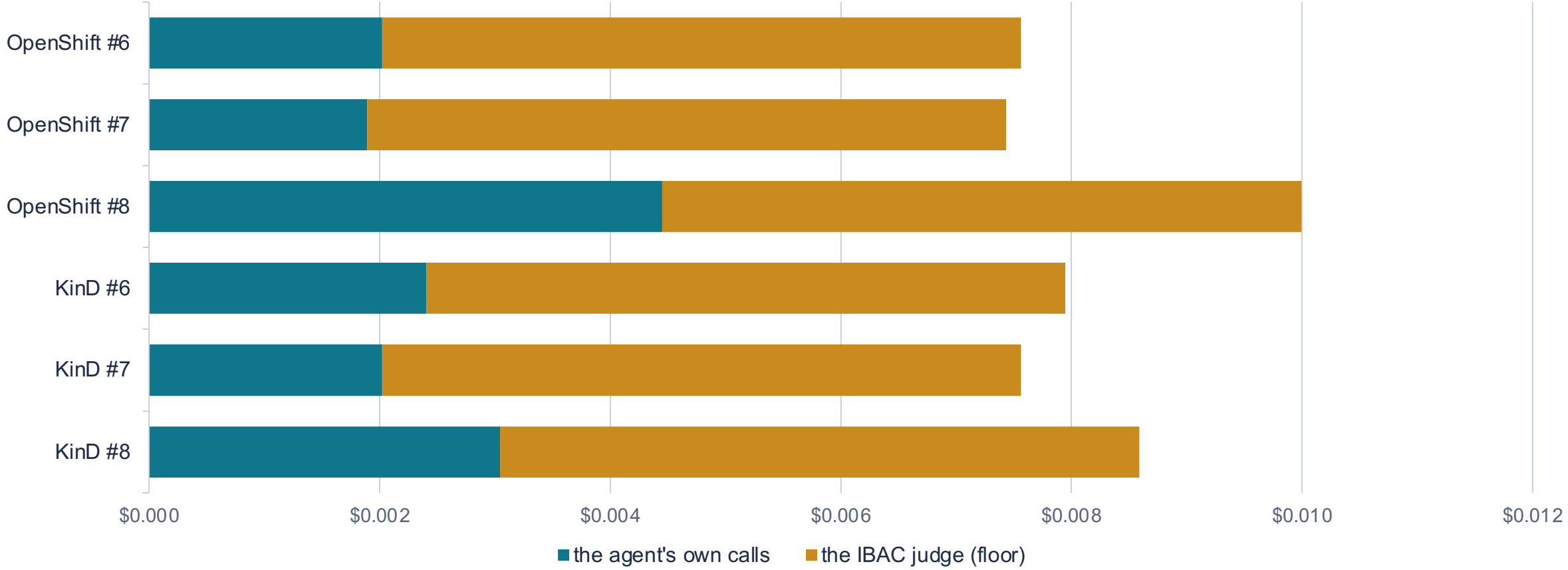
Never quote tokens per task as efficiency

A change that halves your token use and halves your pass rate has gained you nothing. The unbounded row is the honest report of a benchmark that runs and then fails.

Dividing by the pass rate is what turns a token count into an efficiency figure: gsm8k $0.00054 per task becomes $0.00055 per PASS; tau2 $0.1513 per task becomes $0.1815 per PASS — and appworld has no finite cost per success at all, because it passed nothing.

On the plugin legs, the check outcosts the work

Dollars per plugin leg, the agent's own calls and the judge's floor ($0.00111 a call)



One IBAC judge completion costs at least $0.00111 — 2.5× the entire gsm8k task on gpt-5-mini it is authorizing — so across legs #6/#7/#8 the judge bills 1.2–2.9× what the agent does.

The canonical 12-run matrix: one fixed set of request bodies

The same 12 legs on every platform, every version

#	benchmark	tasks	parallel	what it varies
1	gsm8k	1	1	baseline: the smoke test
2	gsm8k	10	1	volume, still serial
3	gsm8k	50	4	volume + concurrency
4	gsm8k	5	4	model swap → Azure/gpt-4.1
5	gsm8k	5	4	preset: auth-only
6	gsm8k	5	4	preset: ibac-only
7	gsm8k	5	4	preset: full (enforce)
8	gsm8k	5	4	full + `ibac:observe`
9	tau2	10	1	multi-turn + user simulator
10	tau2	20	4	multi-turn under concurrency
11	appworld	5	1	long-horizon, gemini-2.5-pro
12	appworld	20	4	long-horizon under concurrency

Why a fixed matrix

- task selection is deterministic: the same leg runs the same tasks anywhere
- so a difference is attributable to the platform, not the workload

Every leg deploys fresh

- only a newly created pod re-pulls `:latest`
- the reports in `docs/results/` derive every number from the mirrored artifacts

Each band of legs changes one thing, and established one thing

So a difference between two legs has one candidate explanation

legs	what they vary	what running them established
#1–#3	gsm8k 1 → 10 → 50 tasks, p=1 → 4	the pipeline is stable and deterministic: 7 of 12 legs have byte-identical input tokens across two unlike clusters
#4	gsm8k on Azure/gpt-4.1, identical tasks	the one clean model comparison: 4.0× the cost of gpt-5-mini for no pass-rate gain
#5–#8	AuthBridge presets, the same 5 tasks	the cost lands in the IBAC judge: +1.36 s/task on OpenShift, +1.48 s on KinD
#9–#10	tau2 10 → 20 tasks, p=1 → 4	multi-turn works end to end; pass rates carry information (0.70–0.90)
#11–#12	appworld 5 → 20 tasks, p=1 → 4	the limits are upstream: 0.00, all three timeouts, ~75% of the bill in two legs

What the matrix as a whole is worth

That the same 12 request bodies measure comparably on two unlike clusters: 0 of 279 rows lost their usage, nothing lost to infrastructure, 9 of 12 pass rates identical.

Read the per-cause table, then the rate

A socket loss and a wrong answer share a denominator. OCP/KinD: timeout 1/2, agent defect 3/2, wrong answer 1/0. Latency travels only approximately (up to 1.8×).

What the 12 runs measured on OpenShift

v1.36 on OpenShift (ykt3): 141 tasks, all 12 legs succeeded

#	bench	pass	err	wall	input tokens
1	gsm8k	1.00	0/1	7 s	320
2	gsm8k	1.00	0/10	64 s	3,166
3	gsm8k	1.00	0/50	80 s	15,684
4	gsm8k	0.80	0/5	21 s	4,404
5	gsm8k	1.00	0/5	9 s	1,564
6	gsm8k	1.00	0/5	16 s	1,564
7	gsm8k	0.80	1/5	16 s	1,564
8	gsm8k	1.00	0/5	14 s	1,564
9	tau2	0.70	0/10	933 s	950,408
10	tau2	0.90	0/20	765 s	1,722,097
11	appworld	0.00	1/5	2,280 s	998,804
12	appworld	0.00	3/20	1,513 s	5,349,737

gsm8k nearly saturates
6 of 8 legs at 1.00; #4 and #7 lose one task each, both the model's own answers.

tau2 is the leg that moves
0.70 and 0.90 here, 0.90 and 0.80 on KinD: at n=10 one task is worth 0.10.

appworld 0.00 is the honest result
Runs complete and tokens record; of its 4 errors, 3 are the agent's own defect, 1 the 600 s budget.

0 tasks lost to infrastructure
An agent LLM call is capped at 120 s (since v1.35), so a stalled call is cut and retried.

9.1 M input, 693 K output tokens
over 5,717 s of run time; appworld is 70% of the input on 24 of 140 rows.

OpenShift and KinD, like for like: seven legs match to the byte

The same 12 request bodies, Service version and instance config

#	bench	pass OCP	pass KinD	input OCP	input KinD
1	gsm8k	1.00	1.00	320	320
2	gsm8k	1.00	1.00	3,166	3,166
3	gsm8k	1.00	1.00	15,684	15,684
4	gsm8k	0.80	0.80	4,404	3,536
5	gsm8k	1.00	1.00	1,564	1,564
6	gsm8k	1.00	1.00	1,564	1,564
7	gsm8k	0.80	1.00	1,564	1,564
8	gsm8k	1.00	1.00	1,564	1,564
9	tau2	0.70	0.90	950,408	965,083
10	tau2	0.90	0.80	1,722,097	1,709,751
11	appworld	0.00	0.00	998,804	911,103
12	appworld	0.00	0.00	5,349,737	5,377,703

9 of 12 pass rates identical

Each delta is one task; wrong answers 1 to 0, agent defects 3 to 2.

7 legs byte-identical on input

320, 3,166, 15,684 and 1,564 ×4: identical work, the strongest like-for-like proof.

#7 differs in pass, not work

Same prompts, different answers: tokens prove the work matched, not that it was right.

Wall time within 11%

5,717 s against 5,165 s; the long legs stay within 40%.

0 lost rows, 0 probe failures

140 rows against 139: KinD's appworld lost one more task to the 600 s budget before its row was written.

What AuthBridge costs: almost all of it is the judge

Non-LLM seconds per task, steady state; the designed 13-leg study, n=50, v1.35

condition	OpenShift	KinD	OCP step	KinD step
baseline (AuthBridge off)	0.168	0.097	—	—
auth-only	0.214	0.130	+0.05	+0.03
ibac-only	1.569	1.610	**+1.36**	**+1.48**
full (auth + ibac)	1.584	1.602	+0.01	−0.01
full + `ibac:observe`	1.583	1.607	−0.00	+0.00

does the finding travel?	OpenShift	KinD	travels?
which layer costs anything	the judge	the judge	yes
serial tool calls judged (p=1)	10 / 10	10 / 10	yes
judged / tool-call ratio, n=50	0.94–1.00	0.96–1.02	yes
tau2 measured / projected	1.66×	2.01×	partly

The judge is the cost, on both clusters

+1.36 s/task on OpenShift, +1.48 s on KinD. The sidecar's presence costs 0.03–0.05 s, every other layer is noise, and with the judge on the two land within 3%.

Still name the cluster

The between-deploy noise floor is 0.02 s on OpenShift and 0.12 s on KinD, and ~0.26 s of each judged call on KinD is the node's DNS being retried.

Why the plugin result can be trusted

The checks behind the numbers on the previous slide

Not a timeout: we checked

A timeout piles values on a round number. Judged tasks run 0.21–9.63 s (OpenShift) and 0.12–7.75 s (KinD) around a ~1.6 s median, no 0.1 s bin above 12%: variable latency, inherited from the judge's own LLM call.

One flag decides whether it measures anything

Every gsm8k leg sends the same 50 prompts, and the gateway replays completions for ~10 min. `BM_CACHE_GAP=900` spaces the legs; the nesting check (no sidecar leg can beat a sidecar-free one) clears by 1.3× on both.

The deploy is the unit of replication

A per-task interval measures variance within one deploy. The honest floor is between two deploys, and only the judge's step clears it: add deploys to rank presets, not tasks.

Two deploys already resolve a 1 s effect

At $\sigma \approx 0.02$ s (OpenShift) and 0.13 s (KinD), so the small steps are genuinely small rather than unresolved.